Single control file for roadmap, Duck checks and migration | v0.2

Atlas - Master Project Reference

Version: 0.2.0 | Date: 2026-10-14 | Owner repository: Swayamjain1240/Atlas

Control status: ACCEPTED PRODUCT/STACK DECISIONS, TARGET ARCHITECTURE; NOT A DECLARATION OF IMPLEMENTATION

1. Mission and product identity

Atlas is a local-first AI-powered Personal Life Operating System: a privacy-preserving index of user-approved personal documents, code and browser-history metadata, queried through local embeddings, RAG and Ollama with verifiable citations.

Success means a user can choose data sources, index them safely, ask source-grounded questions, inspect memories, manage/delete data, and use the UI offline after initial setup.

2. Source-of-truth hierarchy

- 1. This Master Reference: decisions, scope, status, milestones, Duck protocol and change control.
- 2. Requirements: acceptance IDs and release gates (functional, UX, security, non-functional).
- 3. System Design: endpoint contracts, schema, lifecycle, error format and test mapping.
- 4. Architecture: processes, responsibility boundaries, technology and deployment.
- 5. Actual repository code and executed test evidence: ground truth for what is implemented. Documentation NEVER proves implementation.
- For disagreement between these documents and implementation, raise a Duck finding before changing behavior. Preserve git history.

3. Fixed target stack

- Frontend: React 18, Vite, Tailwind CSS, Three.js and GSAP.
- Public backend: Node.js 20+ / Express with auth, validation, rate limiting, API contracts and SQLite ownership.
- Private AI: Python 3.11+ / internal FastAPI, Ollama + llama3.2:3b, Sentence-Transformers all-MiniLM-L6-v2, RAG, ChromaDB ownership.
- Persistence: SQLite metadata/sessions/jobs; ChromaDB vectors; user-approved local files.
- Security: local API keys, internal service authentication, strict localhost/CORS, validation, quotas, safe file access and dependency/secret scans.
- Architecture label: local-first modular multi-process monolith (single product, coordinated local processes).

4. Agreed top-level folders

- backend/ - Express code only after staged migration.
- frontend/ - React/Vite UI, Three.js graph and GSAP animations; reuse existing work.
- aiServices/ - Python AI/ingestion/retrieval stack; private loopback-only worker.
- tests/ - backend, frontend, aiServices, integration, E2E, security and performance.
- docs/ - these four control documents in PDF and editable Markdown formats.
- scripts/ - setup/start/test/audit scripts.
- data/ - local runtime-only SQLite, Chroma, caches; MUST be .gitignored. Prefer OS user-data location.

Single control file for roadmap, Duck checks and migration | v0.2

5. Verified repository baseline (observed 2026-10-14)

- Branch main: 11a6d8102cb0e2fb960ef3a871946101a73c30ac at review time.
- Existing frontend React/Vite/Tailwind views: Chat, Dashboard, Timeline, Settings, Header, Sidebar and a Three.js particle background.
- Existing Python FastAPI backend exposes GET /, GET /api/v1/health and GET /api/v1/status, plus auth/rate-limit/security helpers.
- backend/src/ai/engine.py is a stub: retrieve returns empty list, generate returns warming-up message.
- backend/src/ingestion and backend/src/storage contain only package initializer files; no functional ingestors or database implementation.
- Chat calls POST /api/v1/query and Dashboard calls POST /api/v1/ingest, but those routes do not exist in reviewed main.
- Timeline events are mock data; Dashboard status/memories do not yet represent genuine persisted AI data.
- Git history says 18 smoke tests passed when committed, but no current executable E2E/inference verification was performed during this documentation update.
- Node/Express and aiServices are not implemented in the baseline; these documents specify target changes only.

6. Non-negotiable behavior

- Never silently upload local user content, run cloud inference, enable telemetry, or ingest a new data source without consent.
- Never delete the existing Python backend, installed models or personal data as a shortcut during migration.
- Auth and validation before ingestion, query, settings, memory access and deletion; every protected API call is traceable.
- Answer only using retrieved evidence where feasible; identify insufficient evidence; citations must resolve to real indexed chunks.
- Maintain one-viewport authenticated shell, responsive accessibility and optional reduced motion.
- Avoid changing chosen stack, high-priority requirements or scope without explicit user approval.

7. Development phases (dependencies and exit gates)

Phase 0 - Baseline recovery / branch safety

- Confirm clone, clean git status, env examples and dependency installation; archive v0.1 baseline via Git history/tag or named snapshot.
- Smoke-test existing FastAPI health/status and React build; record failures honestly. No schema/data deletion.
- Exit: baseline and rollback path documented.

Phase 1 - Folder migration and Express foundation

- Create target folders; add Express app, security, config, routing, common errors, SQLite migration skeleton.
- Route frontend dev proxy through Express; Python legacy backend remains available until internal AI worker is ready.
- Exit: React + Express health/status, invalid auth/CORS tests, local-only binding.

Phase 2 - Python AI worker and local inference

- Create aiServices with internal FastAPI auth, Ollama client and embedding model loader; simple mock query works.
- Record model availability offline after downloads, version and CPU/GPU runtime behavior.
- Exit: internal-only health, key rejection, embedding dimension = 384 and deterministic mocked Ollama test.

Phase 3 - Ingestion and persistent storage

- Implement explicit consent, file/PDF/DOCX parsers, safe browser-history metadata reader, chunking, SQLite and Chroma IDs, ingestion jobs.
- Implement safe retries/reconciliation, idempotent re-index and deletions.
- Exit: restart-safe import, search, stats, traceable sources and cleanup tests.

Phase 4 - End-to-end RAG and chat

- Wire query, retrieval, context assembly, Ollama answer and citations; persist optional chat sessions.
- Handle empty index, model unavailable, timeouts and prompt-injection attempts.
- Exit: repeatable React -> Express -> Python -> Chroma -> Ollama query and citation E2E.

Phase 5 - Polished accessible UI

- Add GSAP motion with cleanup/reduced-motion; turn Three.js into real data-driven graph; finish settings, timeline and dashboard.
- Exit: one-viewport behavior and responsive/mobile accessibility, zero placeholder stats/events.

Phase 6 - Hardening and release

- Perform unit, integration, contract, E2E, security and load tests; run offline smoke tests.
- Benchmark against published hardware envelope and document deviations. Verify installation/packaging docs.
- Exit: no critical security/functional failures, backups/restores exercised, release notes and decision log current.

8. Duck checkpoint protocol

- Trigger word: Duck.
- Review the current git branch/head, staged/uncommitted changes, completed phase exit criteria, security controls and docs consistency.
- Output must include: completed work, remaining work, blockers, regression/security risks, decisions required, next safe step and GO / NO-GO.
- Never infer "complete" from a commit message, placeholder route, mock data or documentation.
- Before moving to a new phase, require evidence: tests + manual user-flow demonstration + no unresolved critical issues.

9. Work rules and engineering quality

- Small, meaningful commits. Prefer feature branches and PR review; direct main update only on explicit user request.
- Update requirements/architecture/design if changing interfaces, data schema, consent policy or scope.

- For any file changes, specify CREATE or UPDATE with exact path before implementation; do not generate duplicate masters.
- Test first for bug reproductions; retain regression test with every fix; no ignored test failures.
- Keep secrets/data out of Git. Use env examples with no real credentials. Do not expose private user paths in README/screenshots.
- Preserve original v0.1 PDFs in git history; replace current docs with v0.2, with versions and change rationale.

10. Open risks and explicit decisions

- Privacy: browser-accessible local endpoints need real credentials and origin/Host controls; localhost alone is insufficient.
- Cross-store consistency: SQLite and ChromaDB cannot share an atomic transaction; reconcile partial writes/deletes.
- Performance: p95 <2s full Ollama answer may not be realistic on CPU-only hardware; measure, do not promise.
- Deployment: model downloads may require internet at first install; package fonts and assets to enable truly offline runtime.
- Storage: define backup encryption, OS-specific user-data path and resource quotas before release.
- Product: email, cloud sync, MCP, autonomous agents, Tauri, team mode and calendar are OUT OF SCOPE until approved.

11. Immediate next action

- The documentation update is docs-only; the code in main remains the existing v0.1 scaffold.
- Start Phase 0 review and clone verification, then implement Phase 1 Express foundation without deleting reusable Python modules.
- Verify that the three old v0.1 PDF documents are replaced and a fourth master exists in docs/. Treat these as design baselines, not passing implementation.